Guia de estudo para o exame

O teu trabalho (Proposta 10) explicado com a matéria das aulas 1, 2 e 3. A ideia é usares o código que já escreveste para estudar a matéria: cada conceito das aulas aparece no trabalho, e cada exercício abaixo foi testado, com solução.

Nota: este guia baseia-se nos slides das três aulas. Não sei o formato exato do exame (escrito, em computador, perguntas curtas ou código a escrever). Se tiveres um exame de anos anteriores ou instruções do professor, usa-os para dar prioridade ao que mais conta. O exame é na **sexta-feira, 30 de outubro**, o mesmo dia da entrega no Moodle (até às 18h00) e da apresentação.

1. Mapa: matéria das aulas e onde aparece no teu trabalho

Conceito	Aula	No teu trabalho
Entrada, processamento e saída	1	Entrada: o CSV. Processamento: colunas e resumo. Saída: `sinistros_prioritarios.csv`.
Pseudocódigo e fluxograma	1	A regra `fora_prazo` (comentário no código e Figura 1 do relatório).
Sequência, decisão, repetição	1, 2	Sequência: as 6 tarefas. Decisão: `if` em `cumpre_prazo`. Repetição: `for ... zip`.
Contador e acumulador	1, 2	Versão manual da secção 4 (`total_pendente`); no trabalho, o `groupby` faz o mesmo com `sum` e `count`.
Testar valores-limite	1, 2	Habitacao com 25 dias cumpre (`<=`); 26 não.
Variáveis e tipos	2	int (limites), str (ramo), bool (`fora_prazo`), None (ramo sem limite).
Operadores e comparações	2	`<=`, `==`, `is None`, `or`; no pandas, `|` em vez de `or`.
if / elif / else	2	`cumpre_prazo` e `valor_pendente()`.
for e while	2	`for ramo, dias in zip(...)`. O `while` não é usado, mas sai nas aulas.
f-strings	2	Os `print(f"...")` do programa.
Erros	2	Tiveste um `KeyError` (coluna usada antes de ser criada) e um erro de permissão ao gravar.
Listas, tuplos, dicionários	3	`LIMITES` e `MAPEAMENTO_RAMO` (dicionários); `zip`.
Pandas: dataframe, filtro, groupby	3	Tarefas 1 a 6 (`read_csv`, `isna`, `groupby`, `to_csv`).
Funções	3	`normalizar_ramo`, `limite_regularizacao`, `cumpre_prazo`.
Módulos	3	`from funcoes_auxiliares import ...`.
Classes	3	`Sinistro` com `__init__`, atributos e `valor_pendente()`.
Ficheiros	3	`Path`, `pd.read_csv`, `to_csv(index=False)`.

Atenção ao que NÃO está nos slides. Usaste no trabalho coisas que as aulas não cobrem: *list comprehension* (`[... for ... in ...]`), `lambda`/`.apply`, `.where()`, `.map()`, agregações nomeadas no `agg`, `pd.to_numeric`, `value_counts` e `crosstab`. Para o exame, sabe explicar o equivalente com o que as aulas dão (secção 4): um `for` normal, um `if` e um acumulador.

2. Aula 1: algoritmia no teu trabalho

2.1 Entrada, processamento e saída

Qualquer problema começa por três perguntas: o que entra, o que se faz, o que sai. Para a regra **fora de prazo**:

	Conteúdo
Entrada	ramo (texto) e dias_resolucao (inteiro)

	Conteúdo
Processamento	obter o limite do ramo e comparar os dias com ele
Saída	fora_prazo (lógico: verdadeiro ou falso)
Regras	limite inclusivo; sem limite ou sem dias, não se assinala atraso

2.2 O algoritmo em pseudocódigo (estilo das aulas)

```
Ler ramo, dias
limite <- limite do ramo
Se limite em falta OU dias em falta entao
    cumpre_prazo <- VERDADEIRO
Senao
    cumpre_prazo <- dias <= limite
FimSe
fora_prazo <- NAO cumpre_prazo
Escrever fora_prazo
```

Nas aulas, as condições escrevem-se `Se ... entao ... Senao ... FimSe` e a atribuição com `<-`. Mantém este estilo no exame.

2.3 Simular à mão (trace)

Simular é executar o algoritmo linha a linha, registando as variáveis. Exemplo com **S002** (Habitacao, 32 dias):

Passo	limite	limite em falta?	dias em falta?	dias <= limite	cumpre_prazo	fora_prazo
Depois de obter o limite	25	–	–	–	–	–
Depois das condições	25	Não	Não	32 <= 25: Falso	Falso	–
No fim	25	Não	Não	Falso	Falso	Verdadeiro

Mais casos para treinar (resultados testados):

Ramo	Dias	Limite	cumpre_prazo	fora_prazo
Habitacao	25	25	Verdadeiro	Falso
Habitacao	32	25	Falso	Verdadeiro
Viagem	14	10	Falso	Verdadeiro
Moto (desconhecido)	99	sem limite	Verdadeiro	Falso
Viagem	em falta	10	Verdadeiro	Falso

2.4 Valores-limite

A aula 1 insiste: testa o valor-limite e os imediatamente ao lado. Para Habitacao (limite 25): 24 e 25 cumprem; 26 não cumpre. A frase "até 25 dias" traduz-se por `<=`, e não por `<`. Foi exatamente esta decisão que defendeste no relatório como "limite inclusivo".

3. Aula 2: Python base no teu código

3.1 Variáveis, tipos e operadores

```
LIMITES = {"Automovel": 25, "Habitacao": 25, "Viagem": 10, "ResponsabilidadeCivil": 35}
limite = limite_regularizacao("Moto")     # None   (ausência de valor)
cumpre = dias_resolucao <= limite         # bool   (True ou False)
ramo = "Habitacao"                        # str
dias = 32                                 # int
```

- `=` **atribui;** `==` **compara.** `if x = 5:` dá `SyntaxError`; o correto é `if x == 5:`.
- **Operadores de comparação:** `< <= > >= == !=`. **Lógicos:** `and or not`.

- `is None` é a forma certa de testar "sem valor". Em `cumpre_prazo`: `if limite is None or pd.isna(dias_resolucao):`.

3.2 if, elif e else; a indentação é sintaxe

```
def valor_pendente(self):
    if self.estado == "Pendente":     # a condição termina em :
        return self.valor_reclamado   # bloco indentado (4 espaços)
    else:
        return 0
```

O teu código escreve-o numa linha: `return self.valor_reclamado if self.estado == "Pendente" else 0`. É o mesmo. O `elif` serve para vários caminhos e o Python executa o **primeiro** bloco cuja condição é verdadeira (a ordem importa).

3.3 for, zip e while

```
for ramo, dias in zip(df["ramo"], df["dias_resolucao"]):   # percorre as duas colunas em paralelo
    print(ramo, dias)

saldo = 0
while saldo < 100:      # pergunta obrigatória: o que muda dentro do ciclo?
    saldo = saldo + 25
```

- `for` quando sabes quantas repetições há; `while` quando dependem de uma condição. Um `while` sem nada que altere a condição **nunca termina**.
- `zip` emparelha os elementos da mesma posição; termina na sequência mais curta.
- `range(1, 13)` vai de 1 a 12: o limite final **não** é incluído.

3.4 f-strings e erros comuns

```
print(f"Sinistro: {sinistro.id_sinistro}, Valor Pendente: {sinistro.valor_pendente()}")
print(f"{total:.2f} euros")     # duas casas decimais
```

Erro	Primeira pergunta útil	Exemplo
NameError	O nome foi criado? Está escrito da mesma forma?	print(total) sem total ter sido criado
TypeError	Os tipos são compatíveis?	somar texto com número
IndentationError	Os blocos têm indentação coerente?	linha dentro do if sem recuo
KeyError	A chave/coluna existe nesse momento?	df["valor_pendente"] antes de a criar (o teu erro)
SyntaxError	Falta ':' ou usaste '=' em vez de '=='?	if x = 5:

4. O mesmo programa só com o que as aulas dão

Se o exame pedir código sem pandas, é assim que se escreve a lógica do trabalho usando **listas, dicionários, `for`, `if`, contador e acumulador**. Dados de 5 sinistros (id, ramo, dias, estado, valor reclamado):

```
LIMITES = {"Automovel": 25, "Habitacao": 25, "Viagem": 10, "ResponsabilidadeCivil": 35}
sinistros = [("S001", "Automovel", 18, "Fechado", 3800),
             ("S002", "Habitacao", 32, "Pendente", 6200),
             ("S003", "Viagem", 9, "Fechado", 950),
             ("S004", "ResponsabilidadeCivil", 45, "Pendente", 8400),
             ("S007", "Viagem", 14, "Pendente", 1200)]

total_pendente = 0          # acumulador (inicializado antes do ciclo)
fora_prazo = 0              # contador
for id_, ramo, dias, estado, valor in sinistros:
    if estado == "Pendente":
        total_pendente = total_pendente + valor
    if dias > LIMITES[ramo]:
        fora_prazo = fora_prazo + 1

print("Total pendente:", total_pendente)
print("Fora de prazo:", fora_prazo, "de", len(sinistros))
```

```
Total pendente: 15800
Fora de prazo: 3 de 5
```

Simulação do ciclo (como na aula 1)

Iteração	Sinistro	Estado	dias > limite?	total_pendente	fora_prazo
início	–	–	–	0	0
1	S001	Fechado	18 > 25: Falso	0	0
2	S002	Pendente	32 > 25: Verdadeiro	6200	1
3	S003	Fechado	9 > 10: Falso	6200	1
4	S004	Pendente	45 > 35: Verdadeiro	14600	2
5	S007	Pendente	14 > 10: Verdadeiro	15800	3

Equivalência com o código do trabalho

No trabalho (pandas)	Equivalente com o que as aulas dão
`df["fora_prazo"] = [not cumpre_prazo(r, d) for r, d in zip(...)]`	Um `for` com `zip`, e em cada volta `lista.append(not cumpre_prazo(r, d))`.
`df["valor_pendente"] = df["valor_reclamado"].where(df["estado"] == "Pendente", 0)`	Um `for` com `if estado == "Pendente"`: valor, senão 0.
`df.groupby("ramo")["valor_pendente"].sum()`	Um dicionário usado como acumulador: `totais[ramo] = totais.get(ramo, 0) + valor`.
`df[(df["estado"] == "Pendente") | df["fora_prazo"]]`	Um `for` com `if estado == "Pendente" or fora`, acrescentando à lista de prioritários.

Porque se usa `|` no pandas e não `or`: `or` compara dois valores únicos; numa coluna inteira é preciso `|` (ou) e `&` (e), com parênteses à volta de cada condição.

5. Aula 3: estruturas de dados, pandas, funções, módulos e classes

5.1 Listas, tuplos e dicionários

Estrutura	Sintaxe	Quando usar	No trabalho
Lista	`[]`	sequência alterável, valores do mesmo tipo	listas criadas pelo `zip` e pelo `.tolist()`
Tuplo	`()`	sequência que não deve mudar	os pares devolvidos pelo `zip`
Dicionário	`{}`	pares chave-valor, acesso por nome	`LIMITES`, `MAPEAMENTO_RAMO`

- Índices começam em **0**; `lista[-1]` é o último. `len()`, `sum()`, `min()`, `max()`, `.append()`.

- Dicionário: `d["chave"]` dá erro (`KeyError`) se a chave não existir; `d.get("chave", valor)` devolve o valor por defeito. O teu `MAPEAMENTO_RAMO.get(valor, valor)` usa isto.
- Percorrer: `for chave, valor in d.items():`.

5.2 Pandas: o que cada operação do trabalho faz

Operação	O que faz
`pd.read_csv(csv_file)`	Lê o CSV para um dataframe (tabela).
`df.dtypes` / `df.isna().sum()` / `df.duplicated().sum()` / `.unique()`	Tipos / valores em falta por coluna / linhas repetidas / valores distintos.
`df["nova"] = ...`	Cria uma coluna calculada.
`df[df["x"] > 4000]`	Filtra linhas (a condição dá uma coluna de verdadeiro/falso).
`df.groupby("canal")["receita"].sum()`	Agrupa por categoria e soma. É o padrão da aula 3.
`.fillna(0)` / `.dropna()`	Preencher ou remover valores em falta (a escolha depende do significado).
`df.sort_values(by=[...], ascending=[False, False])`	Ordena por uma ou mais colunas.
`df.to_csv("f.csv", index=False)`	Grava o CSV; `index=False` evita a coluna 0,1,2...

5.3 Funções, módulos e classes

```
def limite_regularizacao(ramo):          # ramo: PARÂMETRO
    return LIMITES.get(normalizar_ramo(ramo))

limite_regularizacao("Viagem")           # "Viagem": ARGUMENTO  ->  10

from funcoes_auxiliares import cumpre_prazo      # módulo = ficheiro .py com funções

class Sinistro:                          # molde
    def __init__(self, id_sinistro, ramo, estado, valor_reclamado):
        self.id_sinistro = id_sinistro   # atributos guardados no objeto
        self.estado = estado
        self.valor_reclamado = valor_reclamado

    def valor_pendente(self):            # método: usa self
        return self.valor_reclamado if self.estado == "Pendente" else 0

s = Sinistro("S002", "Habitacao", "Pendente", 6200)   # instância
s.valor_pendente()                                     # 6200
```

- **Parâmetros** estão na definição; **argumentos** são os valores passados na chamada; `return` entrega o resultado.
- `__init__` corre ao criar o objeto; `self` é o próprio objeto; cada instância tem os seus atributos.
- Módulos separam responsabilidades: as regras ficam no `funcoes_auxiliares.py` e o script principal importa-as.

6. Exercícios com solução (estilo das aulas)

Exercício 1: função com if / elif / else

Escreve `classificar_atraso(ramo, dias)` que devolva “dentro do prazo” se não houver excesso face ao limite, “atraso ligeiro” se o excesso for até 5 dias (inclusive) e “atraso grave” se for superior. Testa os valores-limite.

```
def classificar_atraso(ramo, dias):
    excesso = dias - limite_regularizacao(ramo)
    if excesso <= 0:
        return "dentro do prazo"
    elif excesso <= 5:
        return "atraso ligeiro"
    else:
        return "atraso grave"

classificar_atraso("Habitacao", 25)   # dentro do prazo   (excesso 0)
classificar_atraso("Habitacao", 30)   # atraso ligeiro    (excesso 5, inclusive)
classificar_atraso("Habitacao", 31)   # atraso grave      (excesso 6)
classificar_atraso("Viagem", 14)      # atraso ligeiro    (excesso 4)
```

Exercício 2: dicionário como contador

Com a lista `sinistros` da secção 4, conta quantos sinistros há por ramo.

```
n_por_ramo = {}
for id_, ramo, dias, estado, valor in sinistros:
    if ramo in n_por_ramo:
        n_por_ramo[ramo] = n_por_ramo[ramo] + 1
    else:
        n_por_ramo[ramo] = 1
print(n_por_ramo)
```

```
{'Automovel': 1, 'Habitacao': 1, 'Viagem': 2, 'ResponsabilidadeCivil': 1}
```

Exercício 3: while com sentinela (simulação)

```
valores = [4, 7, 9, 0, 5]
i = 0
soma = 0
while valores[i] != 0:
    soma = soma + valores[i]
    i = i + 1
print(soma, i)
```

Simula: a soma acumula 4, 11 e 20 e o ciclo pára quando encontra o 0, na posição 3. O 5 nunca é lido. **Resultado:** `20 3`**.** Se tirasses `i = i + 1`, o ciclo nunca terminaria.

Exercício 4: acrescentar um método à classe

Acrescenta à classe `Sinistro` um método `esta_pendente()` que devolva `True` ou `False`.

```
class Sinistro:
    def __init__(self, id_sinistro, ramo, estado, valor_reclamado):
        self.id_sinistro = id_sinistro
        self.ramo = ramo
        self.estado = estado
        self.valor_reclamado = valor_reclamado

    def valor_pendente(self):
        return self.valor_reclamado if self.estado == "Pendente" else 0

    def esta_pendente(self):            # NOVO: indentado dentro da classe, com self
        return self.estado == "Pendente"

Sinistro("S002", "Habitacao", "Pendente", 6200).esta_pendente()   # True
Sinistro("S001", "Automovel", "Fechado", 3800).esta_pendente()    # False
```

Exercício 5: pandas

Com o dataframe `df` do trabalho (colunas `valor_pendente` e `fora_prazo` já criadas), responde. Resultados testados:

Pergunta	Código	Resultado
Valor pendente por canal?	`df.groupby("canal")["valor_pendente"].sum()`	Agente 8 400; Corretor 15 000; Digital 7 400

Pergunta	Código	Resultado
Que sinistros estão fora de prazo?	`df[df["fora_prazo"]]["sinistro"]`	S002, S004, S005, S007, S008, S009, S012
Quantos fora de prazo por ramo?	`df.groupby("ramo")["fora_prazo"].sum()`	Automovel 2; Habitacao 1; RC 3; Viagem 1
Pendentes com valor reclamado > 6 000?	`df[(df["estado"] == "Pendente") & (df["valor_reclamado"] > 6000)]`	S002, S004, S012
Total pendente?	`df["valor_pendente"].sum()`	30 800

Exercício 6: encontra o erro

Código	Erro	Correção
`print(df["valor_pendente"].sum())` antes da linha que cria a coluna	KeyError: a coluna ainda não existe	Cria a coluna primeiro e só depois a usa.
`if estado = "Pendente":`	SyntaxError: `=` atribui	`if estado == "Pendente":`
`print(total)` sem `total` criado	NameError	Inicializa a variável antes (por exemplo `total = 0`).
`df.to_csv("f.csv")`	Grava o índice 0,1,2... como coluna	`df.to_csv("f.csv", index=False)`
`total = total + valor` dentro do ciclo, sem `total = 0` antes	NameError (acumulador não inicializado)	`total = 0` antes do `for`.
`df[df["a"] > 1 and df["b"] < 5]`	and não funciona em colunas	Usa `&` com parênteses: `df[(df["a"] > 1) & (df["b"] < 5)]`

7. Perguntas de teoria e respostas curtas

O que distingue um algoritmo de um programa?

O algoritmo é a ideia e os passos da solução (em linguagem natural, fluxograma ou pseudocódigo); o programa é a implementação numa linguagem concreta, como o Python.

O que é um bom algoritmo?

Preciso (sem ambiguidade), finito (termina) e testável (podemos verificar a saída).

Quais são as três estruturas fundamentais?

Sequência, decisão e repetição.

Contador e acumulador?

O contador conta ocorrências (`n = n + 1`); o acumulador soma valores (`total = total + x`). Ambos têm de ser inicializados antes do ciclo.

Porque se testam valores-limite?

É onde se erra mais: confirma se “até X” é `<=` ou `<`. Testa X e os valores imediatamente ao lado.

Diferença entre `=` e `==`?

= atribui um valor a uma variável; == compara dois valores e devolve verdadeiro ou falso.

Lista, tuplo e dicionário?

Lista: sequência alterável; tuplo: sequência que não se altera; dicionário: pares chave-valor, acedidos pela chave.

Quando usar for e quando while?

for quando se sabe quantas repetições há; while quando depende de uma condição (e algo tem de a alterar).

O que é um dataframe?

Uma tabela com linhas (observações) e colunas (variáveis), cada coluna com o mesmo tipo de dados.

O que faz o groupby?

Divide as linhas em grupos por uma categoria, aplica uma operação (soma, média, contagem) a cada grupo e junta o resultado.

Parâmetro ou argumento?

Parâmetro: o nome na definição da função; argumento: o valor passado na chamada.

O que são `__init__` e `self`?

__init__ é chamado ao criar o objeto e inicializa os atributos; self é o próprio objeto, que permite aceder aos seus atributos e métodos.

Porque separar o código em módulos?

Para dividir por responsabilidades: é mais simples de ler, testar e reutilizar.

Porque `index=False` no to_csv?

Para não gravar o índice interno do pandas como uma coluna extra sem nome.

8. Folha de consulta rápida

```
# DECISÃO                                 # REPETIÇÃO
if cond:                                  for x in lista:
    ...                                       ...
elif outra:                               for a, b in zip(l1, l2):
    ...                                       ...
else:                                     while cond:
    ...                                       ...   # algo tem de alterar cond

# FUNÇÃO                                  # CLASSE
def nome(parametro):                      class Nome:
    return resultado                          def __init__(self, a):
                                                  self.a = a
# MÓDULO                                      def metodo(self):
from ficheiro import funcao                       return self.a

# PANDAS
df = pd.read_csv(Path("f.csv"))           df["c"] = df["a"] - df["b"]
df[df["c"] > 0]                           df.groupby("g")["v"].sum()
df.isna().sum()                           df.to_csv("o.csv", index=False)
```

Plano até sexta-feira

- **Terça e quarta:** lê as secções 2 e 3 e faz os traces à mão, sem olhar para as soluções.
- **Quinta de manhã:** faz os exercícios da secção 6 sozinho e só depois compara com as soluções; treina escrever pseudocódigo no estilo das aulas.
- **Quinta à tarde:** revê a secção 7 em voz alta e as tuas dúvidas sobre o trabalho (`lambda`, comprehension, `groupby`).

- **Sexta:** o exame, a apresentação e a entrega no Moodle (18h00) são no mesmo dia. Submete o trabalho **antes**, de preferência na quinta à noite.